

Laplaciano em Grafos aplicado ao pipeline DINOv2 + KNN

Trabalho de Teoria Espectral

June 11, 2026

1 Objetivo

Este documento descreve, passo a passo, a matemática implementada no código `Code/Core/Laplaciano_Propagacao/propagando_rotulos.py`. O pipeline parte de imagens, gera embeddings com DINOv2, constrói um grafo KNN ponderado e aplica um método espectral baseado no Laplaciano normalizado.

No código atual, a configuração fixa principal é:

`DATASET_NAME = CUB_Cleaned50, K = 20, NUM_CLUSTERS = 50.`

Observação 1. Apesar do nome “propagação de rótulos”, o código atual realiza *clustering espectral não supervisionado*. Os rótulos reais entram apenas no final, para avaliar a qualidade dos clusters encontrados.

2 Dados e embeddings

Considere um conjunto de imagens

$$\mathcal{I} = \{I_1, I_2, \dots, I_n\}.$$

No caso atual do CUB limpo,

$$n = 2889.$$

Cada imagem é passada por uma rede DINOv2. O resultado é um vetor de atributos

$$x_i \in \mathbb{R}^{384}.$$

Agrupando todos os vetores, obtemos a matriz de embeddings

$$X = \begin{bmatrix} - & - & x_1^\top & - & - \\ - & - & x_2^\top & - & - \\ & & \vdots & & \\ - & - & x_n^\top & - & - \end{bmatrix} \in \mathbb{R}^{n \times 384}.$$

No código, esta matriz é salva em:

`DataSets/Emb/CUB_Cleaned50_emb.pt`

e também em formato NumPy:

`DataSets/Emb/CUB_Cleaned50_emb.npy.`

3 Listas ranqueadas por vizinhança

Para cada imagem i , o código usa uma estrutura `BallTree` para encontrar os vizinhos mais próximos no espaço dos embeddings.

Assim, para cada i , temos uma lista ranqueada:

$$R_i = (i, r_{i,1}, r_{i,2}, \dots, r_{i,m}),$$

onde o primeiro elemento é normalmente a própria imagem, e $r_{i,1}$ é o vizinho mais próximo de i , $r_{i,2}$ é o segundo mais próximo, e assim por diante.

Essas listas são salvas no arquivo:

`DataSets/Runs/CUB_Cleaned50_dinov2_vits14_output.json`.

4 Construção do grafo KNN ponderado

Definição 1 (Grafo ponderado). *Um grafo ponderado é uma tripla*

$$G = (V, E, W),$$

onde V é o conjunto de vértices, E é o conjunto de arestas e W contém pesos não negativos associados às arestas.

No nosso caso, cada imagem é um vértice:

$$V = \{1, 2, \dots, n\}.$$

O código escolhe os K primeiros vizinhos de cada lista ranqueada. Para $K = 20$, cada imagem escolhe seus 20 vizinhos mais próximos.

Construímos primeiro uma matriz direcionada $W^{dir} \in \mathbb{R}^{n \times n}$. Se j aparece na posição r da lista de vizinhos de i , então:

$$W_{ij}^{dir} = \frac{1}{r}.$$

Logo, o primeiro vizinho recebe peso 1, o segundo recebe peso 1/2, o terceiro recebe peso 1/3, etc.

No código:

$$W[i, j] = \max(W[i, j], 1.0/\text{rank}).$$

Depois, o grafo é tornado não direcionado por simetrização:

$$W = W^{dir} + (W^{dir})^\top.$$

Essa escolha faz com que relações recíprocas fiquem mais fortes. Por exemplo, se i escolhe j com peso 1, e j escolhe i com peso 1/2, então:

$$W_{ij} = W_{ji} = 1 + \frac{1}{2} = 1.5.$$

5 Grau dos vértices

O grau ponderado de um vértice i é a soma dos pesos das arestas ligadas a ele:

$$d_i = \sum_{j=1}^n W_{ij}.$$

A matriz diagonal dos graus é:

$$D = \begin{bmatrix} d_1 & 0 & \cdots & 0 \\ 0 & d_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & d_n \end{bmatrix}.$$

No código, não é necessário montar D explicitamente. Basta calcular o vetor

$$d_i^{-1/2} = \frac{1}{\sqrt{d_i}}.$$

6 Laplaciano normalizado

O Laplaciano normalizado usado no código é:

$$L = I - D^{-1/2} W D^{-1/2}.$$

Em coordenadas, a matriz normalizada de adjacência é:

$$S_{ij} = \frac{W_{ij}}{\sqrt{d_i d_j}}.$$

Assim,

$$L = I - S.$$

No código, isto aparece como:

```
normalized_adjacency = d_inv_sqrt[:, None] * W * d_inv_sqrt[None, :]
```

e depois:

```
L = np.eye(n_samples) - normalized_adjacency.
```

Observação 2. A normalização por $D^{-1/2}$ reduz o efeito de vértices com grau muito alto. Assim, o espectro do grafo passa a refletir melhor a estrutura de comunidades, e não apenas a quantidade de conexões de cada vértice.

7 Autovetores e embedding espectral

A ideia central é usar os autovetores associados aos menores autovalores do Laplaciano. Calculamos:

$$L u_\ell = \lambda_\ell u_\ell,$$

com

$$\lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_c,$$

onde $c = \text{NUM_CLUSTERS}$.

No caso atual:

$$c = 50.$$

Formamos a matriz

$$U = \begin{bmatrix} | & | & \cdots & | \\ u_1 & u_2 & \cdots & u_c \\ | & | & \cdots & | \end{bmatrix} \in \mathbb{R}^{n \times c}.$$

Cada linha de U representa uma imagem no espaço espectral:

$$U_{i,:} \in \mathbb{R}^c.$$

8 Normalização das linhas

Seguindo a forma usual do algoritmo de Ng–Jordan–Weiss, o código normaliza cada linha de U :

$$Y_{i,:} = \frac{U_{i,:}}{\|U_{i,:}\|_2}.$$

Assim, a matriz final usada pelo KMeans é:

$$Y \in \mathbb{R}^{n \times c}.$$

Intuitivamente, o KMeans passa a comparar mais a direção dos pontos no espaço espectral do que o tamanho dos vetores.

9 Clustering com KMeans

O KMeans é aplicado nas linhas de Y , não nos embeddings originais do DINOv2:

$$\text{cluster}(i) = \text{KMeans}(Y_{i,:}).$$

O resultado é um vetor:

$$z \in \{0, 1, \dots, c-1\}^n,$$

onde z_i é o cluster atribuído à imagem i .

10 Rótulos reais e avaliação

Para o dataset CUB_Cleaned50, os rótulos reais são lidos de:

`DataSets/CUB_Cleaned50/manifest.csv`.

O CUB usa classes numeradas de 1 até 50. No código, elas são convertidas para:

$$0, 1, \dots, 49.$$

Seja y_i o rótulo real da imagem i . Como os nomes dos clusters do KMeans são arbitrários, não podemos comparar diretamente $z_i = y_i$.

Por isso, construímos uma matriz de confusão:

$$C_{ab} = \#\{i : z_i = a \text{ e } y_i = b\}.$$

Depois usamos o algoritmo Húngaro para encontrar o melhor pareamento um-para-um:

$$\pi(a) = b.$$

A acurácia reportada é:

$$\text{acc} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{\pi(z_i) = y_i\}.$$

Além disso, o código calcula:

ARI = Adjusted Rand Index,

e

NMI = Normalized Mutual Information.

11 Arquivos gerados

O script salva os principais resultados em `DataSets/Laplacian`. Com a configuração atual, o prefixo de saída é:

`CUB_Cleaned50_laplacian_label_propagation_k20_c50.`

Arquivo	Conteúdo
<code>_clusters.npy</code>	Cluster bruto do KMeans para cada imagem
<code>_mapped_labels.npy</code>	Cluster convertido para o melhor rótulo real
<code>_embedding.npy</code>	Matriz espectral Y
<code>_eigenvalues.npy</code>	Autovalores escolhidos do Laplaciano
<code>_confusion.npy</code>	Matriz de confusão cluster $\times$ classe
<code>.json</code>	Grafo com posições, arestas, clusters e acertos
<code>.png</code>	Grafo colorido por cluster
<code>_correctness.png</code>	Grafo colorido por acerto/erro
<code>_embedding.png</code>	Projeção das duas primeiras coordenadas espectrais

12 Resumo do algoritmo

1. Ler as listas ranqueadas de vizinhos do arquivo JSON.
2. Construir a matriz ponderada W^{dir} usando pesos $1/r$.
3. Simetrizar a matriz: $W = W^{dir} + (W^{dir})^\top$.
4. Calcular os graus $d_i = \sum_j W_{ij}$.
5. Construir o Laplaciano normalizado:

$$L = I - D^{-1/2} W D^{-1/2}.$$

6. Calcular os c menores autovetores de L .
7. Montar a matriz espectral U e normalizar suas linhas, obtendo Y .
8. Rodar KMeans nas linhas de Y .
9. Ler os rótulos reais do manifest.
10. Usar o algoritmo Húngaro para mapear clusters para classes.
11. Salvar métricas, arrays, JSON e figuras.

13 Comentário final

O código implementa um método espectral sobre um grafo de similaridade entre imagens. O DINOv2 fornece uma representação inicial das imagens; o KNN converte essa representação em um grafo; o Laplaciano extrai a estrutura global do grafo; e o KMeans transforma as coordenadas espectrais em clusters discretos.